

Математические основы алгоритмов, осень 2020 г.
Лекция 3: Метод динамического программирования: задача
о порядке умножения матриц. Символьные строки.
Нахождение наибольшей общей подпоследовательности:
«народный» алгоритм, алгоритм Хиршберга. Поиск в
ориентированном графе: поиск в ширину, поиск в глубину,
топологическая сортировка*

Александр Охотин

21 ноября 2020 г.

Содержание

1	Метод динамического программирования (окончание)	1
1.1	Задача о порядке умножения матриц	1
2	Нахождение наибольшей общей подпоследовательности	3
2.1	Строки	3
2.2	«Народный» алгоритм	3
2.3	Алгоритм Хиршберга	5
3	Поиск в ориентированном графе	6
3.1	Поиск в ширину	6
3.2	Поиск в глубину	7
3.3	Топологическая сортировка	8

1 Метод динамического программирования (окончание)

1.1 Задача о порядке умножения матриц

Пусть нужно перемножить n матриц $M_1 \times \dots \times M_n$, где каждая матрица M_i имеет размер $m_{i-1} \times m_i$, для набора чисел $m_0, m_1, \dots, m_n$. В силу ассоциативности, скобки можно расставить как угодно. От их расстановки зависит общее число операций, и, чтобы умножить матрицы быстрее, ставится задача заранее определить наилучший порядок их умножения.

*Краткое содержание лекций, прочитанных студентам 1-го курса СПбГУ, обучающимся по программам «Математика» и «МАиАД», в осеннем семестре 2020–2021 учебного года. Страница курса: http://users.math-cs.spbu.ru/~okhotin/teaching/algorithms_2020/.

Пример 1. Для произведения трёх матриц размера 100×2 , 2×100 и 100×3 , если догадаться сперва перемножить две последних матрицы, то получится матрица 2×3 , которую легко будет перемножить с матрицей 100×2 — всего потребуется выполнить $2 \cdot 100 \cdot 3 + 100 \cdot 2 \cdot 3 = 1200$ умножений и примерно столько же сложений. Если же начать с умножения двух первых матриц, получится неудобноумножаемая промежуточная матрица размера 100×100 , и общее число умножений составит $100 \cdot 2 \cdot 100 + 100 \cdot 100 \cdot 3 = 50000$.

Конечно, можно просто перебрать все возможные порядки — их ровно число Каталана C_n — для каждого определить общее число операций, и найти среди этих чисел наименьшее. Но не исключено, что чем за такое браться, быстрее будет перемножить матрицы как придётся!

Задача решается методом динамического программирования. Сперва ставится цель: построить верхнедиагональную числовую матрицу T размера $n \times n$, где $T_{i,j}$ — наименьшее число действий, необходимых для вычисления $M_{i+1} \times \dots \times M_j$.

Внешний цикл по длине куска $\ell = j - i$, второй — по i , во внутреннем перебираются все разбиения $M_{i+1} \times \dots \times M_j = (M_{i+1} \times \dots \times M_k) \times (M_{k+1} \times \dots \times M_j)$ и вычисляется следующее значение.

$$T_{i,j} = \min_{k=i+1}^{j-1} (T_{i,k} + T_{k,j} + m_i m_k m_j)$$

Алгоритм 1 Нахождение наилучшего порядка умножения матриц и их умножение в соответствии с этим порядком

На входе: матрицы $M_1, \dots, M_n$, размер каждой матрицы $M_i = m_{i-1} \times m_i$.

Нахождение числа действий

```

1: for  $i = 1$  to  $n$  do
2:    $T_{i-1,i} = 0$  /* произведение одной матрицы — это она сама, считать нечего */
3: for  $\ell = 2$  to  $n$  do
4:   for  $i = 0$  to  $n - \ell$  do
5:     пусть  $j = i + \ell$  /* ищется наилучший порядок для  $M_{i+1} \times \dots \times M_j$  */
6:      $T_{i,j} = \infty$  /* пока не найдено никаких способов перемножить */
7:     for  $k = i + 1$  to  $j - 1$  do
8:        $t = T_{i,k} + T_{k,j} + m_i m_k m_j$  /* способ перемножить за столько шагов */
9:       if  $t < T_{i,j}$  then /* если это быстрее известного */
10:         $T_{i,j} = t$  /* ... то оценка числа действий улучшается */

```

Произведение $M_{i+1} \times \dots \times M_j$ вычисляется следующей процедурой.

Процедура умножить(i, j)

```

1: if  $i + 1 = j$  then
2:   return  $M_j$ 
3: for  $k = i + 1$  to  $j - 1$  do
4:   if  $T_{i,j} = T_{i,k} + T_{k,j} + m_i m_k m_j$  then /* для какого-то  $k$  равенство выполнится */
5:     return умножить( $i, k$ )  $\times$  умножить( $k, j$ )

```

Потом, используя таблицу, сами матрицы перемножаются вызовом процедуры умножить($0, n$).

Время работы: $O(n^3)$ для построения таблицы, далее — $2n - 1$ вызовов процедуры умножить(i, j), в каждом — $O(n)$ итераций цикла. И ещё само время умножения матриц.

В процедуре умножить(i, j) можно обойтись без цикла по k , если записывать значение $k_{i,j}$ при построении таблицы.

Ww дубль-вэ
 Ww дубль-вэ (курсив)
 Ωω омега



Рис. 1: (слева) Различия между начертаниями букв w (дубль-вэ) и ω (омега); (справа) Каждый раз, когда строку обозначают буквой ω , умирает котёнок.

2 Нахождение наибольшей общей подпоследовательности

Ставится задача сравнить две строки на похожесть. Пример: сравнение двух редакций одного и того же текста. Другой пример: ДНК двух организмов. Реализовано в программе `diff`. Как это работает?

2.1 Строки

Сперва задаётся конечное множество *символов* Σ , называемое *алфавитом*. В абстрактных примерах элементы Σ обозначаются строчными латинскими буквами из начала алфавита ($a, b, \dots$).

Определение 1. Строкой над алфавитом Σ называется всякая конечная последовательность $w = a_1 \dots a_\ell$, где $\ell \geq 0$, и $a_1, \dots, a_\ell \in \Sigma$ — символы.

Строки обычно обозначаются латинскими буквами w (дубль-вэ¹), u, v, x, y и z . Например, $w = abb$ — это 3-символьная строка над алфавитом, содержащим символы a и b .

Число символов в строке называется её *длиной*, $|w| = \ell$. Существует единственная строка длины 0, называемая *пустой строкой* и обозначаемая через ϵ . Множество всех строк над алфавитом Σ обозначается через Σ^* .

Строка, полученная удалением 0 и более символов из строки, при сохранении относительного порядка оставшихся символов, называется её *подпоследовательностью*: то есть, если $1 \leq i_1 < \dots < i_k \leq m$, то строка $a_{i_1} \dots a_{i_k}$ — подпоследовательность строки $a_1 \dots a_m$.

2.2 «Народный» алгоритм

Пусть $a_1 \dots a_m$ и $b_1 \dots b_n$ — две строки. Требуется найти самую длинную общую подпоследовательность этих двух строк. Обычно бывает нужна даже не сама подпоследовательность — а последовательность номеров соответствующих позиций в первой и второй строках. Эта последовательность называется *выравниванием* двух строк. «Наивный» алгоритм: перебрать все варианты совмещения подпоследовательностей. Но это будет экспоненциальное время относительно m и n .

Решается динамическим программированием за время $O(mn)$; это «народный» алгоритм, его автора трудно установить — и это классический пример применения метода динамического программирования. Для всякого префикса (начального куска) $u_i = a_1 \dots a_i$ первой строки и для всякого префикса $v_j = b_1 \dots b_j$ второй строки алгоритм находит *длину* наибольшей общей подпоследовательности u_i и v_j и запоминает её в переменной $T_{i,j}$. Ячейки прямоугольной таблицы $T_{i,j}$ заполняются последовательно, начиная от более коротких префиксов, и заканчивая префиксами u_m и v_n — то есть, данными строками целиком.

¹На худой конец, «дабл-ю». Но, ради Бога, ни в коем случае не «омега»! Омега (ω, Ω) — это совсем другая буква, и строки ею никогда не обозначаются! (см. рис. 1)

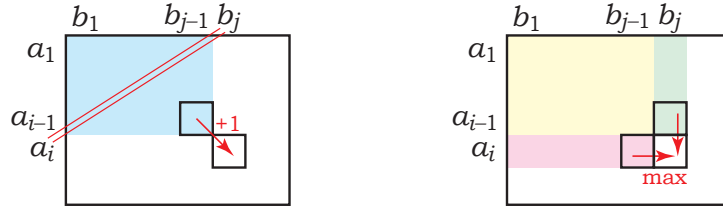


Рис. 2: Построение элемента $T_{i,j}$: (слева) случай $a_i = b_j$; (справа) случай $a_i \neq b_j$.

Значение каждой ячейки $T_{i,j}$ вычисляется так. Если $i = 0$ или $j = 0$, то есть, один из двух префиксов пуст, то $T_{i,j} = 0$. Для всяких $i, j \geq 1$ рассматриваются следующие два случая. Если последние символы двух префиксов совпадают ($a_i = b_j$), это значит, что в одной из наибольших общих подпоследовательностей они соответствуют друг другу: действительно, если a_i соответствует другому символу $b_{j'}$, где $j' < j$, то тогда символам $b_{j'+1}, \dots, b_j$ не соответствует ничего, и потому можно переключить a_i на b_j , получив другую общую подпоследовательность u_i и v_j той же длины; если же ни a_i , ни b_j ничему не соответствуют, то их можно поставить друг другу в соответствие и получить более длинную общую подпоследовательность. Поэтому можно считать, что a_i и b_j соответствуют друг другу, а перед ними идёт наибольшая общая подпоследовательность u_{i-1} и v_{j-1} длины $T_{i-1,j-1}$. Если же последние символы различны ($a_i \neq b_j$), то в наибольшей общей подпоследовательности или a_i не соответствует никакому символу (и тогда остаётся подпоследовательность длины $T_{i-1,j}$), или b_j не соответствует никакому символу (остаётся $T_{i,j-1}$). На основе этих соображений строится следующая формула.

$$T_{i,j} = \begin{cases} T_{i-1,j-1} + 1 & \text{если } a_i = b_j \\ \max(T_{i-1,j}, T_{i,j-1}) & \text{если } a_i \neq b_j \end{cases}$$

Алгоритм 2 Нахождение длины наибольшей общей подпоследовательности

На входе: строки $u = a_1 \dots a_m$ и $v = b_1 \dots b_n$. Строится матрица T размера $(m+1) \times (n+1)$, где $T_{i,j}$ — длина наибольшей общей подпоследовательности $u_i = a_1 \dots a_i$ и $v_j = b_1 \dots b_j$.

```

1: for  $j = 0$  to  $n$  do
2:    $T_{0,j} = 0$ 
3: for  $i = 1$  to  $m$  do
4:    $T_{i,0} = 0$ 
5:   for  $j = 1$  to  $n$  do
6:     if  $a_i = b_j$  then
7:        $T_{i,j} = T_{i-1,j-1} + 1$ 
8:     else
9:        $T_{i,j} = \max(T_{i-1,j}, T_{i,j-1})$ 
10: return  $T_{m,n}$ 

```

Сама наибольшая общая подпоследовательность и соответствующее ей совмещение исходных строк строятся на основе чисел в таблице, от конца к началу. Глядя на $T_{i,j}$ и на символы a_i и b_j , можно определить, по какому из двух случаев вычислено значение $T_{i,j}$.

Недостаток алгоритма в том, что таблица занимает память $O(mn)$, и это *много*. Если нужно только вычислить *длину* наибольшей общей подпоследовательности, то хранить всю таблицу не обязательно, можно ограничиться двумя столбцами (или двумя строчками). Но обычно бывает нужна сама подпоследовательность, а скорее даже совмещение двух строк,



Рис. 3: Дэниэл Хиршберг.

дающее эту подпоследовательность. Можно ли использовать меньше памяти и при этом всё это вычислить? Оказывается, что можно.

2.3 Алгоритм Хиршберга

Алгоритм Хиршберга [1975]: построение наибольшей общей подпоследовательности за время $O(mn)$, используя память $O(\min(m, n))$.

Этот алгоритм интересен красивым сочетанием двух изучавшихся методов построения алгоритмов: используется метод «разделяй и властвуй», внутри которого наилучшее разбиение находится подобно динамическому программированию.

Пусть $u = u'u''$ — некоторое разбиение u . Тогда оптимальное совмещение u и v совмещает u' с каким-то начальным куском v — пусть это v' — и u'' — с остатком, v'' . Нужно найти это разбиение $v = v'v''$, чтобы потом отдельно запустить совмещение двух соответствующих пар кусков.

Как это сделать? Алгоритм делит u на две подстроки u', u'' примерно равной длины. Сперва динамическим программированием находится последняя строчка таблицы $T^{u', v}$, соответствующей алгоритму LCS для u' и v . Её j -й элемент содержит длину наибольшей общей подпоследовательности u' и $v_j = b_1 \dots b_j$. Аналогично находится последняя строчка таблицы $T^{(u'')^R, v^R}$, её j -й элемент — длина наибольшей общей подпоследовательности u'' и $v_j = b_{j+1} \dots b_n$. Дальше остаётся сложить эти суммы поэлементно и посмотреть, где достигается максимум — это и будет искомое разбиение $v = v'v''$. Для этого вычисления алгоритм использовал $O(|v|)$ ячеек памяти, которые теперь можно освободить.

Далее алгоритм дважды рекурсивно вызывает сам себя, чтобы вычислить наилучшее совмещение u' и v' , и u'' и v'' . Полученные совмещения последовательно приписываются друг к другу.

Лемма 1. Алгоритм Хиршберга работает за время $O(mn)$.

Доказательство. Принимая за единицу времени время, затрачиваемое на вычисление значения одного элемента $T_{i,j}$ в «народном» алгоритме, утверждается, что в общей сложности будет выполнено не более чем $2mn$ шагов.

Пусть $f(m, n)$ — время работы в наихудшем случае. Тогда индукцией по m и n доказывается неравенство $f(m, n) \leq 2mn$. При запуске на строках u и v , где $|u| = m$ и $|v| = n$, вычисление таблицы $T_{u', v}$ займёт $\frac{1}{2}mn$ шагов, и за столько же шагов будет вычисляться таблица $T^{(u'')^R, v^R}$. После этого проводятся два рекурсивных вызова, один из которых занимает

$f(\frac{m}{2}, k)$ шагов, а другой — $f(\frac{m}{2}, n - k)$ шагов, для некоторого k . Время работы рекурсивных вызовов оценивается по предположению индукции, откуда получается оценка того же вида для $f(m, n)$.

$$f(m, n) = 2 \cdot \frac{1}{2} mn + \max_k (f(\frac{m}{2}, k) + f(\frac{m}{2}, n - k)) \leq mn + \max_k (mk + m(n - k)) = 2mn$$

□

3 Поиск в ориентированном графе

3.1 Поиск в ширину

Поиск в ширину (breadth-first search): в ориентированном графе $G = (V, E)$ обойти все вершины, достижимые из данной вершины $s \in V$, каждую по одному разу. Это программа, которую то и дело приходится писать на практике: граф может быть задан неявно, вершины могут быть сложными объектами, рёбра — какими-то действиями над этими объектами, и нужно получить все объекты, выражимые из данного через имеющиеся действия. Или же граф может быть непосредственно задан в памяти: каждая вершина — это массив указателей на другие такие же вершины, соответствующих исходящим дугам.

Алгоритм поиска в ширину работает так. В каждый момент времени каждая вершина может быть помечена или не помечена. Если вершина помечена, это значит, что алгоритм уже нашёл путь из s в неё. Кроме пометок на вершинах, алгоритм хранит очередь, в которой находятся все те помеченные вершины, для которых ещё не обработаны исходящие дуги. Таким образом, в каждый момент времени каждая вершина может быть или (1) непомеченной, или (2) помеченной, но необработанной, или (3) помеченной и обработанной.² Изначально помечена только вершина s и очередь состоит из s . На каждом шаге из очереди извлекается следующая вершина u , рассматриваются все исходящие из неё дуги (u, v) , и если v оказывается непомеченной, она помечается и помещается в конец очереди.

Алгоритм 3 Поиск в ширину

Структуры данных: очередь вершин для обработки, множество всех встреченных вершин (можно хранить вместе с вершинами как пометки, а если граф задан неявно, то в отдельной структуре данных).

```

1: пометить  $s$ 
2: поместить  $s$  в конец очереди
3: while очередь не пуста do
4:     извлечь из очереди первый элемент,  $u$ 
5:     for all  $v: (u, v) \in E$  do
6:         if  $v$  не помечена then
7:             пометить  $v$ 
8:             поместить  $v$  в конец очереди
9:     теперь  $u$  обработана

```

Время работы: каждая вершина попадает в очередь не более одного раза, каждая выходящая из неё дуга рассматривается однажды. Отсюда: $O(|E|)$.

²Их иногда называют «белыми», «серыми» и «чёрными», соответственно.

Утверждение 1. В каждый момент времени очередь состоит из некоторых вершин, находящихся на расстоянии ℓ от s , вслед за которыми идут некоторые вершины, находящиеся на расстоянии $\ell + 1$ от s . При этом все вершины на расстоянии, меньшем, чем ℓ , уже обработаны, равно как и все вершины на расстоянии ℓ , не вошедшие в очередь. Из вершин на расстоянии $\ell + 1$ в очереди есть ровно все потомки обработанных вершин.

Набросок доказательства. Базис: в очереди только s , она находится на расстоянии 0, и это все вершины на расстоянии 0.

Переход: пусть извлекается u , она на расстоянии ℓ . Все новые вершины v , добавленные в очередь по дуге (u, v) , будут на расстоянии $\ell + 1$. Действительно, будь для v более короткий путь, она, по предположению индукции, уже была бы помечена ранее. $\square$

Утверждение 2. (I) алгоритм помечает вершину v тогда и только тогда, когда есть путь из s в v ; (II) если алгоритм находит v по дуге (u, v) , то один из кратчайших путей из s в v идёт через u ; (III) все пройденные дуги (u, v) образуют дерево.

Доказательство. (I)

(обходит $\Rightarrow$ есть путь) индукция по длине вычисления. Базис: помечена в этой строке — тогда это s . Переход: когда v помечается, есть ранее помеченная вершина u и дуга (u, v) . Путь из s в u есть по предположению индукции, он продолжается дугой до искомого.

(есть путь $\Rightarrow$ обходит) индукция по длине пути. Базис: 0, помечается в первой строке. Переход: пусть u — предпоследняя вершина на пути из s в v . Тогда путь в неё короче, и по предположению индукции u когда-то помечается. Одновременно u заносится в очередь, откуда она рано или поздно будет извлечена, и тогда при рассмотрении дуги (u, v) будет обнаружена вершина v .

(II) Из утверждения 1.

(III) Всякий раз, когда алгоритм переходит по дуге, он переходит в ранее не рассматривавшуюся вершину. Поэтому все рассмотренные дуги образуют ориентированный граф, в котором у одной вершины степень захода 0, а у остальных — 1. В таком графе не может быть циклов, в том числе неориентированных. $\square$

В ходе своей работы алгоритм проходит по некоторому остовному дереву подграфа, достижимого из s . Для каждой пройденной дуги можно добавить какое-нибудь содержательное действие — например, записывать куда-нибудь это остовное дерево или вычислять кратчайшее расстояние от s до очередной вершины.

Если нужно обойти не только вершины, достижимые из данной, а вообще все вершины, то надо перезапустить $BFS(s)$ на непомеченных вершинах, пока таковые будут оставаться.

3.2 Поиск в глубину

Поиск в глубину (depth-first search, DFS): пройти до конца самый длинный путь, затем вернуться на шаг назад и продолжить следующий путь — и так далее, пока все вершины, достижимые из данной, не будут найдены.

В каждый момент времени каждая вершина — непомеченная («белая»); помеченная и обрабатываемая («серая») или помеченная и обработанная («чёрная»). Изначально все непомеченные. По мере работы алгоритма каждая вершина будет сперва *обнаруживаться* и помечаться (начало обработки), а потом в какой-то момент её обработка будет завершаться. Сперва произвольно выбирается одна непомеченная вершина: она помечается, и все её

непомеченные соседи один за другим рекурсивно обходятся. Когда обходы всех соседей заканчиваются и последний рекурсивный вызов возвращается, обработка текущей вершины завершается, процедура возвращается. По завершении обхода выбирается следующая непомеченная вершина, и повторяется то же самое.

Алгоритм 4 Поиск в глубину

Процедура $DFS(u)$

```

1: пометить  $u$  /* «серая» */
2: for all  $v: (u, v) \in E$  do
3:     if  $v$  не помечена then /* «белая» */
4:          $DFS(v)$ 
5: обработка  $u$  закончена /* «чёрная» */

```

Обрабатываемые, «серые» вершины — это те, которые находятся в стеке возврата.

Алгоритм 5 Поиск в глубину

Основная процедура

```

1:  $t = 0$ 
2: for all  $s \in V$  do
3:     if  $s$  не помечена then
4:          $DFS(s)$ 

```

Процедура $DFS(u)$

```

1: пометить  $u$ 
2:  $t = t + 1$ 
3:  $\text{начало}(u) = t$ 
4: for all  $v: (u, v) \in E$  do
5:     if  $v$  не помечена then
6:          $DFS(v)$ 
7:  $t = t + 1$ 
8:  $\text{конец}(u) = t$ 

```

Если записать все дуги, по которым делаются рекурсивные вызовы, то получится лес.

Кроме того, алгоритм часто используется, чтобы поставить каждой вершине в соответствие два числа, соответствующие времени её обхода, измеряемому в шагах работы алгоритма. В момент перекраски в серый вершине назначается *время обнаружения*, а в момент перекраски в чёрный — *время окончания*.

Время работы: $O(|V| + |E|)$; написать просто $|E|$ нельзя, потому что может быть много изолированных вершин.

3.3 Топологическая сортировка

Дан ориентированный граф $G = (V, E)$. Задача: расположить его вершины в таком порядке, чтобы для каждой дуги $(u, v) \in E$ вершина u была перечислена раньше, чем v . Это возможно тогда и только тогда, когда граф ациклический.

Пример: порядок чтения книг. Не прочитав книгу по математическому анализу, не имеет смысла читать книгу о дифференциальных уравнениях — это дуга в графе. Топологическая сортировка даёт допустимый порядок прочтения.

Решение: запустить поиск в глубину, потом отсортировать вершины по времени окончания. Законченные позже всего должны быть в начале.

Почему это работает? Это следует из простых свойств поиска в глубину.

Лемма 2. *Граф ациклический тогда и только тогда, когда при поиске в глубину никогда не рассматривается дуга, ведущая в вершину, находящуюся в стеке возврата (дуга из «серой» вершины в «серую»).*

Доказательство. Поскольку последовательность вершин в стеке возврата образует путь в графе, такая дуга замкнёт этот путь в цикл.

И обратно: если есть цикл, то какая-то из его вершин, v , будет рассмотрена первой. Тогда рекурсивные вызовы, делаемые процедурой $DFS(v)$, обойдут весь цикл и рассмотрят замыкающее цикл ребро. $\square$

Лемма 3. *Если в ациклическом графе есть дуга (u, v) , то время завершения v меньше, чем время завершения u .*

Доказательство. Эта дуга однажды будет рассматриваться. В этот момент v нет в стеке возврата. Если v помечена, то у неё уже есть время завершения, а у u ещё только будет (позднее). Если же v ранее не рассматривалась, то в неё спустится рекурсия, и по возвращении из рекурсии v будет уже обработана — а u только предстоит получить время завершения. $\square$

Список литературы

[1975] D. S. Hirschberg, “A linear space algorithm for computing maximal common subsequences”, *Communications of the ACM*, 18:6 (1975), 341–343.